



HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



TRƯỜNG ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Designing a Dynamic Semantic Cache for LLM-Based Access Control

ONE LOVE. ONE FUTURE.

Problem statement: Modern Access Control

- **Growing Regulatory Pressure:** Sensitive data (e.g., in healthcare, finance), legally compliant protection mechanisms.
- **The Expertise Gap:** Domain experts but lack the technical skill to implement formal security models.
- **Implementation Risks:** misinterpreted security goals, system vulnerabilities and inconsistencies.
- **Limitations of Traditional Models (RBAC, ABAC):**
 - Ambiguity: Difficulty processing subjective requests (e.g., "close relative").
 - Dynamic Contexts: Inability to swiftly adapt to edge cases (e.g., medical emergencies).
 - Scalability: High overhead when recomputing complex rules for repeated access requests.

Motivation

LLMs can address these gaps by:

- Interpreting natural language requests and generating semantic embeddings
- Comparing new requests against a semantic cache of past decisions
- Adapting to fuzzy contexts (e.g., “urgent”, “trusted colleague”)

Objectives

- Design a semantic cache that stores embeddings of access requests/contexts
- Reduce decision latency for repeated requests • Improve accuracy for ambiguous requests
- Detect anomalies (e.g., cache poisoning attacks)

Input - Output

Table 1: System Inputs

Type	Example	Format
Access Request	“I’m the on-call nurse; I need patient X’s medical record.”	Natural language text
Context	Time: 3:00 AM, Location: ER, Role: Nurse, History: 5 prior approved accesses	JSON/Structured
Access Rules	“Grant if (role = health-care_professional AND time > 10PM AND time < 6AM)”	Formal logic (XACML)
Feedback	Audit logs: “Access denied at 3:05 AM for record Y (reason: role mismatch)”	Structured (CSV/JSON)

Table 2: System Outputs

Type	Example	Format
Decision	{"access": "granted", "confidence": 0.92, "reason": "emergency context validated"}	JSON
Cache Update	Store request embedding + decision + context	Vector DB (FAISS)
Alert	“Anomaly detected: High similarity to known privilege escalation attack (score: 0.87)”	Notification



Canonical Request Contract

Every request is normalized before processing.

- {"request_id": "uuid",
- "timestamp_utc": "ISO-8601",
- "user": {"user_id": "string", "role": "string", "department": "string", "region": "string", "clearance_level": "int"},
- "context": {"ip_address": "string", "device_id": "string", "session_id": "string", "mfa_state": "passed|failed|unknown", "incident_state": "normal|elevated|critical"},
- "resource": {"resource_type": "document|table|api|tool", "resource_id": "string", "sensitivity": "public|internal|restricted|confidential"},
- "query": {"prompt": "string", "purpose": "string"}

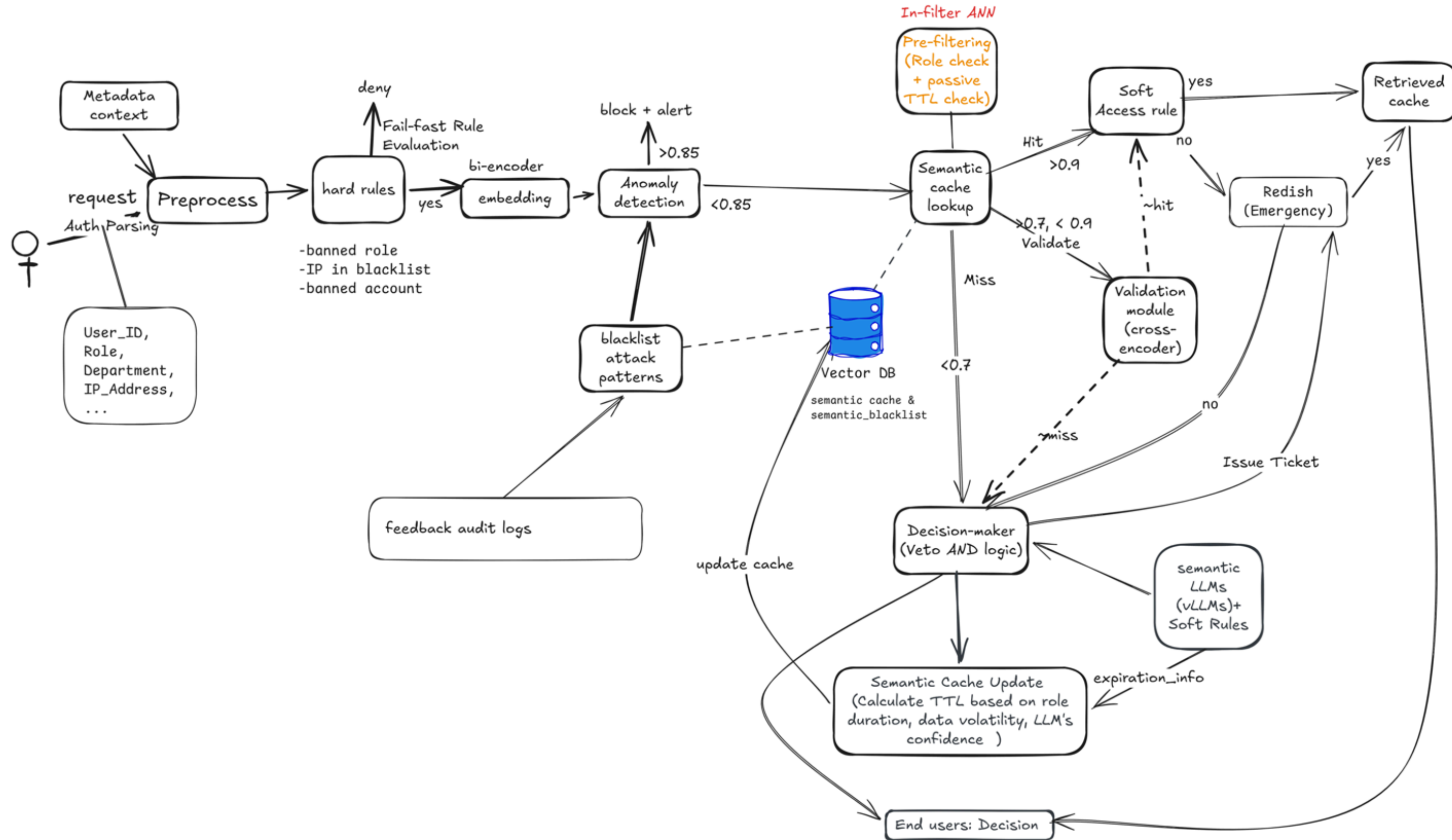
Synthetic Data

- {"scenario_type": "Standard Hit", "Inputs": {"Access_Request": "I need to access the trademark registration dossier for Vinamilk from 2023.", "Context": {"Time": "9:00 AM", "Location": "HQ Office", "Role": "legal_auditor", "History": "5 prior approved accesses"}, "Access_Rules": "Grant if (role == 'legal_auditor' AND resource_type == 'trademark_dossier' AND entity == 'Vinamilk' AND year == '2023')", "Feedback": "None"}, "Outputs": {"Decision": {"access": "granted", "confidence": 0.98, "reason": "Request matches policy for legal auditor"}, "Cache_Update": "Store request embedding + decision + context", "Alert": "None"}, "request_id": "req_f00703bd"}

SLO Targets

- Authorization decision latency:
- P50 $\leq$ 120 ms on cache hit.
- P95 $\leq$ 400 ms mixed traffic.
- Security:
- False allow rate $< 0.5\%$.
- 100% audit log coverage for final decisions.

Proposed Architecture



Phase A: Ingestion and Hard Deny Gate

1. Parse identity, network, and resource metadata.
 2. Evaluate deterministic hard rules:
 - Blocked users, blocked IP/CIDR, banned role-resource pairs.
 - Mandatory controls not satisfied (MFA missing, expired session).
 3. If any hard rule fails, return DENY_HARD immediately and log.
- Output: allow_to_semantic_pipeline = true|false

Phase B: Embedding and Threat Similarity

1. Generate query embedding using a fast bi-encoder.
2. Compare against a vector collection of known malicious/abusive patterns.
3. If ``threat_similarity >= T_attack``, return ``DENY_THREAT`` and trigger alert.

Default: ``T_attack = 0.85`` in balanced mode.

→ Output: ``safe_for_cache_lookup = true|false``.

Phase C: Semantic Cache Retrieval with In-Filter ANN

1. Search vector DB with metadata filters applied inside ANN query:
 - `role`, `department`, `region`, `clearance\_level`, `resource\_type`.
 - `not\_expired` and optional policy version compatibility.
2. Compute best-match score `S\_cache`.
3. Route by score:
 - `S\_cache >= T\_hit` -> direct cache-hit path to soft access rules (skip cross-encoder).
 - `T\_validate\_low <= S\_cache < T\_hit` -> validation path.
 - `S\_cache < T\_validate\_low` -> LLM decision path.

Default thresholds in balanced mode:

- `T\_hit = 0.90`
- `T\_validate\_low = 0.70`

Phase D: Validation and Soft Access

For uncertain matches only ($T_validate_low \leq S_cache < T_hit$):

1. Cross-encoder validates query against retrieved cached intent/answer/policy tuple.
2. If validated ($\sim hit$), route to the same soft access rule path used by direct cache hits.
3. Soft access checks include transient constraints:
 - Active incident posture.
 - Temporary policy exceptions.
 - Rate anomaly on user/session.

Emergency branch:

- If soft checks fail but a valid emergency ticket exists, allow cached response with `ALLOW_EMERGENCY`.
- Otherwise escalate to LLM decision path.

Phase E: LLM Decision-Maker (Veto Logic)

1. Compose policy-aware prompt with structured facts only.
2. LLM proposes `allow|deny|escalate` plus reason.
3. Deterministic post-check vetoes any LLM output violating hard policy.
4. Return final decision:
 - `ALLOW`
 - `DENY`
 - `ESCALATE\_HUMAN`

Phase F: Post-Decision Updates

1. Write decision and explanation to immutable audit stream.
2. If eligible, update semantic cache with:
 - Embedding, normalized intent, decision artifact, policy version.
3. Apply dynamic TTL:
$$\text{TTL} = \text{base_ttl}(\text{role}, \text{sensitivity}) * \text{confidence_factor} * \text{policy_stability_factor}$$
4. Feed suspicious/incorrect outcomes into blacklist retraining pipeline.

Tech Stack

- Gateway & Orchestration:
 - Framework: FastAPI
 - NLU (extract entities/ intent): GLiNER, Spacy
 - Add metadata context: JWT + Redis
- Static Rules & Anomaly Detection:
 - Static Policy Engine: OPA (Open Policy Agent)
- Anomaly / Blacklist: Bloom Filter
- AI - Inference (Bi-Encoder, Cross-Encoder & LLM):
 - Tokenizer & Embeddings: bi-encoder (BGE-m3) running by ONNX Runtime
 - Cross-Encoder in Validation Module: AAI/bge-reranker-base running by ONNX Runtime
 - Semantic LLMs: vLLMs (Llama-3-8B-Instruct or Qwen-2.5-7B)

Tech Stack

- Semantic Cache Core:
 - Vector Database: Qdrant
 - Cache Management Framework: custom (... based on GPTCache -- suvey later)
- Event-Driven & Background Process
 - Message Broker: RabbitMQ
 - Background Jobs / Passive TTL Cleanup: Celery

Stage 1. Design Vector DB

- Logic Data Model: semantic_cache_entry, attack_pattern_entry

Field	Type	Required	Purpose
role	string	yes	Access context filter
department	string	yes	Access context filter
region	string	yes	Access context filter
clearance_level	integer	yes	Access context filter
resource_type	string	yes	Resource scoping filter
policy_version	string	yes	Prevent reuse across policy changes
cached_text	string	yes	Cached request text for validation reranking
cached_decision	string	yes	Stored previous decision outcome
ttl_seconds	integer	yes	TTL metadata for observability/debugging
expires_at_ts	integer	yes	Active expiration filter
expires_at_utc	string	yes	Human-readable expiration timestamp

Semantic Cache Payload Schema

Field	Type	Required	Purpose
source	string	yes	Pattern provenance
reason_code	string	yes	Why the prompt was marked suspicious
prompt	string	yes	Original suspicious text

Attack Pattern Payload Schema

Stage 2. Design Policy Schema

- The policy layer includes 3 logical stages: hard_allow, soft_allow, and veto

Prototype Results

Settings:

- Current rules:
 - Deny if ``context.mfa_state != "passed"```
 - Deny if ``context.session_id == ""```
 - Allow otherwise

Prototype Results

Scenario	Wall Time (Avg / P95)	App Latency (Avg / P95)	Total Cost (USD)	Total Tokens	Cache Score	Cross Score
Cache Miss(LLM Fallback)	2346.9 ms / 3292.2 ms	2215.3 ms / 3097.0 ms	\$0.00204	10,304	0.0	0.0
Validation Band(Cross-Encoder)	130.6 ms / 329.8 ms	104.0 ms / 311.0 ms	\$0.00000	0	0.856	0.999
Cache Hit(Direct Cache)	92.0 ms / 193.5 ms	58.7 ms / 99.0 ms	\$0.00000	0	1.0	0.0

Table 1. Semantic Cache Performance Summary
(Based on 30 iterations per scenario)



Prototype Results

Scenario	Avg	Min	P50 (Median)	P95	Max
App Latency: Cache Miss	2215.3 ms	1422.0 ms	2072.0 ms	3097.0 ms	3654.0 ms
App Latency: Validation	104.0 ms	62.0 ms	80.0 ms	311.0 ms	390.0 ms
App Latency: Cache Hit	58.7 ms	34.0 ms	49.0 ms	99.0 ms	275.0 ms

Table 2. App Latency Distribution Breakdown
(Based on 30 iterations per scenario)



Prototype Results

Scenario	Wall Time (Average)	Wall Time (P50 / Median)	Wall Time (P95)	Total API Cost	Avg Cost per Request
Cache Disabled (Baseline)	2673.89 ms	1988.31 ms	6594.99 ms	\$0.061323	\$0.000061
Cache Enabled	226.75 ms	85.43 ms	622.21 ms	\$0.002476	\$0.000002
Impact / Improvement	11.79x Faster	23.27x Faster	10.60x Faster	95.96% Reduction	-

Table 3. Performance & ROI Summary (1,000 Requests)
(40 unique clusters, 960 paraphrased requests)



A large graphic on the left side of the slide. It features a dark blue background with a pattern of red dots of varying sizes arranged in concentric, slightly irregular circles, creating a sense of depth and movement. The word "HUST" is centered within this graphic.

HUST

THANK YOU !